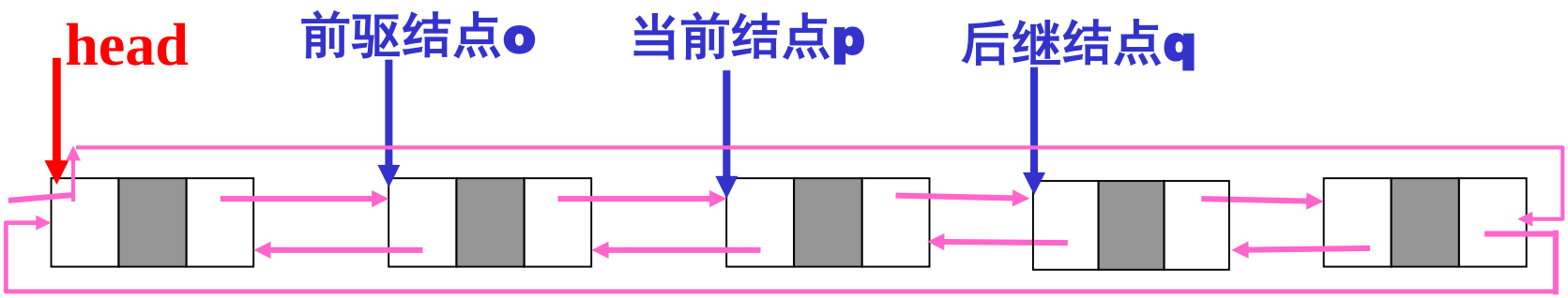
2.2、双向循环链表



p->llink=o　p->rlink=q

p->llink->rlink?

```
template<typename T> class DblList;
template<typename T> class DblNode{
        T info;
        DblNode<T> *llink,*rlink
public:
        DblNode(T data);
        DblNode();
        T Getinfo(){return info;};
        friend class DblList<T>;
};
```

双向链结点模板

信息科学与工程学院
王文博 2026

2.2 、双向循环链表

```
template<typename T>class DblList{
        DblNode<T> *head,*current;
public:
        DblList();
        ~DblList();
        void Insert(const T & data);
        DblNode<T>* Remove(DblNode<T>* p);
        void Print();
        int Length();     //计算链表长度
        DblNode<T> *Find(const &T data) const ;  //查找数据
        void MakeEmpty();  //清空链表
        //其它操作略
};
```

双向循环链模板

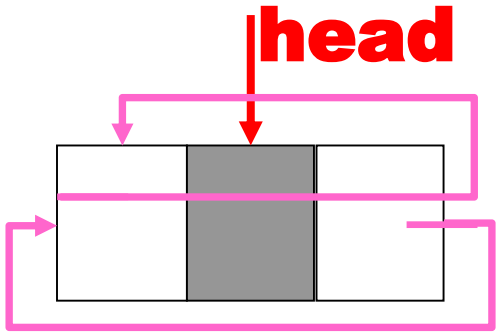
2.2、双向循环链表

带表头空表建立

```
template<typename T> DblList<T>::DblList()
{
        head=new DblNode<T>();
        head->rlink=head->llink=head;
        current=NULL;
}
```

申请结点空间，返回给head指针

左、右链接指针指向自己



2.2、双向循环链表

插入结点

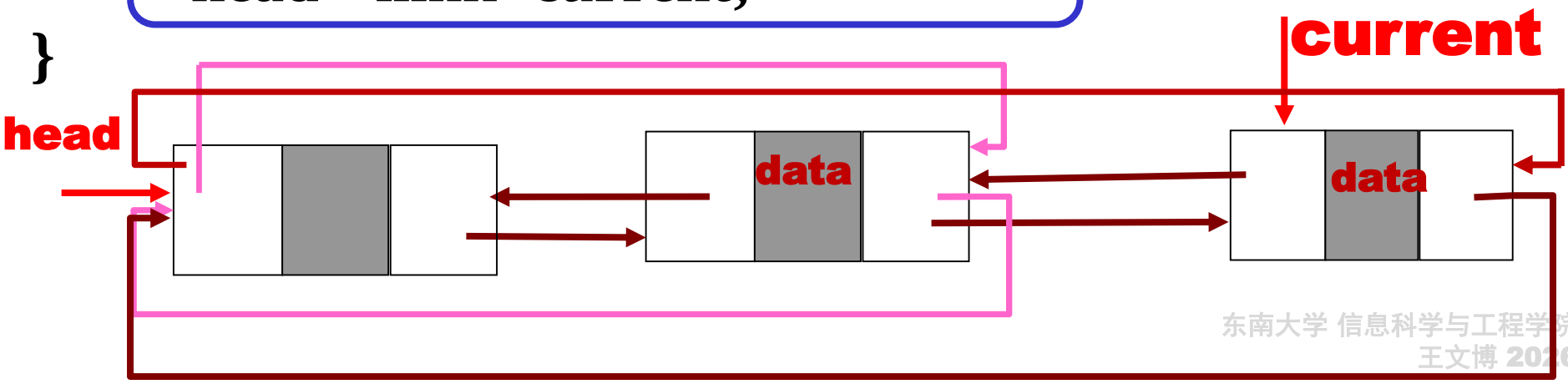
```
template<typename T>
 void DblList<T>::Insert(const T & data)
{
        current=new DblNode<T>;
        current->info= data;
        current->rlink=head;
        current->llink=head->llink;
        head->llink->rlink=current;
        head->llink=current;
}
```

申请结点空间，返回给current指针

存入数据

current左、右链接指针

循环链接



2.2、双向循环链表

删除结点

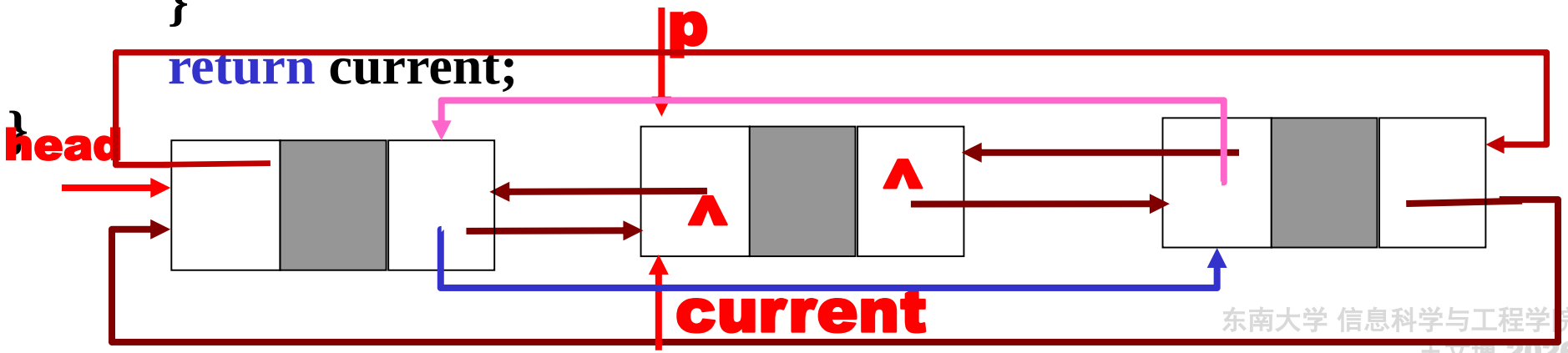
```
template<typename T>
 DblNode<T>* DblList<T>::Remove(DblNode<T>* p)
{
        current=head->rlink;
        while(current!=head&&current!=p)
                current=current->rlink;
        if(current==head) current=NULL;
        else{
                p->llink->rlink=p->rlink;
                p->rlink->llink=p->llink;
                p->rlink=p->llink=NULL;
        }
        return current;
}
```

查找要删除结点

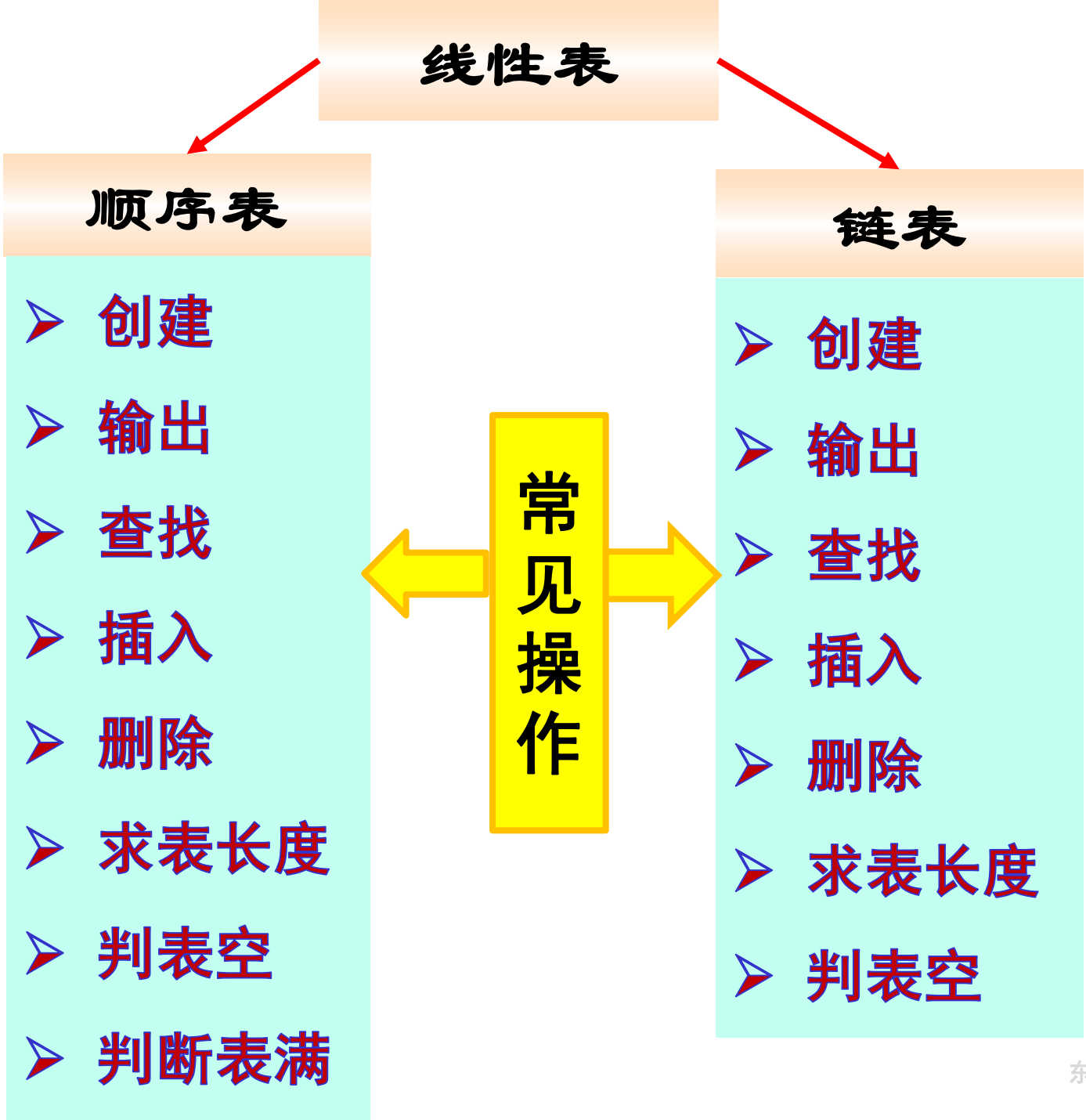
修改p前驱结点的右链接指针

修改p后继结点的左链接指针

清空p指针的左右链接指针



By 王文博



东南大学 信息科学与工程学院
王文博 2026

By 王文博

栈与队列

东南大学 信息科学与工程学院
王文博 2026

3、栈与队列

- 栈和队都是特殊的线性表，限制存取位置的线性结构。
- 实现方式：
  - 顺序表
  - 链表

3.1、栈

- 栈的基本概念：

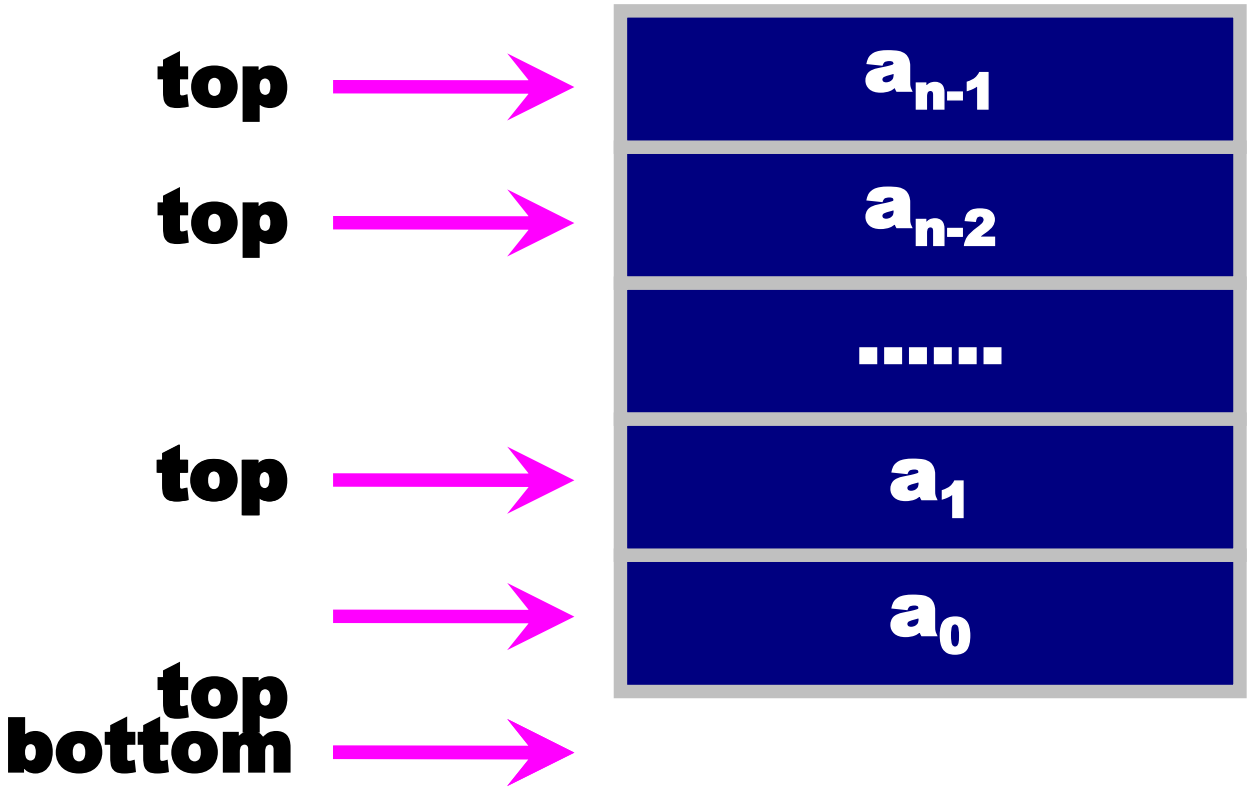
  只允许在表的一端进行插入和删除的线性表。

  允许进行插入和删除的一端叫做栈顶(top)，而另一端叫栈底(bottom)。

- 特点：

  后进先出（LIFO：Last In First Out）的线性表

3.1.1、栈的基本操作



顺序栈示意图

- top与bottom相同就是空栈
- 只有栈顶与操作有关

3.1.1、栈的基本操作



1. 一个栈的初始状态为空。现将元素1、2、3、4、5、A、B、C、D、E依次入栈，然后再依次出栈，则元素出栈的顺序是 ？

2. 若进栈的序列为1，2，3，4，进栈过程中可以出栈，则下列不可能的一个出栈序列是（ ）

A. 1，4，3，2

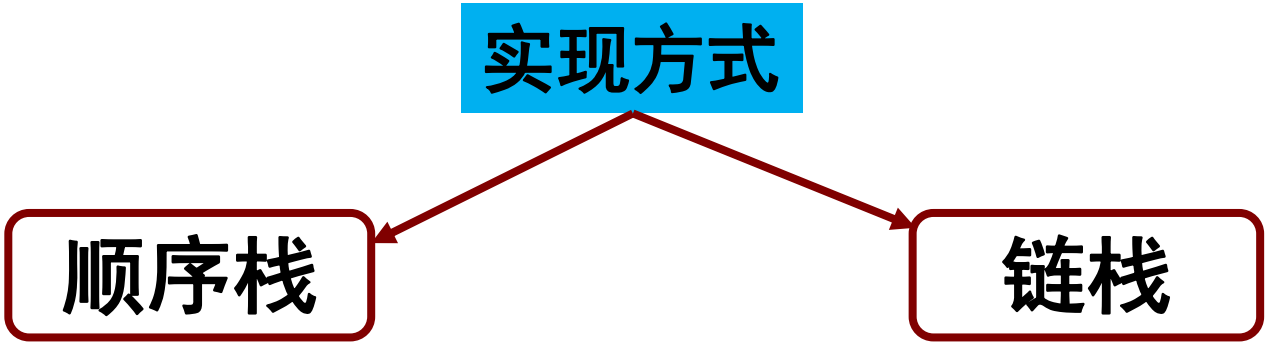
B. 2，3，4，1

C. 3，1，4，2

D. 3，4，2，1

3.1.1、栈的基本操作

- 压栈
- 弹出
- 取元素
- 清空栈元素



3.1.2 、顺序栈的类模板

```
template<typename T>class Stack
{
        int top;                    //栈顶指针（下标）
        T *elements;                //动态建立的元素
        int maxSize;                //栈最大容纳的元素个数
public:
         Stack(int=20);             //构造函数，默认设为20元素
         ~Stack() ;
         void Push(const T &data);  //压栈
        T& Pop();                   //弹出，top--
        T& GetElem(int i) const;    //取数据，top不变
        void MakeEmpty() ;                          //清空栈
         bool IsEmpty() const ;                     //判栈空
        bool IsFull() const ;
        void PrintStack() const;    //输出栈内所有数据
};
```

3.1、顺序栈的类模板

```
template<typename T>
Stack<T>::Stack(int maxs)
{
   maxSize=maxs;
   top=-1;
   elements=new T [maxSize];
    assert(elements!=0);
}
```

构造函数

初始空栈

动态建立栈空间

分配不成功结束程序

assert(expression)的作用是在**Debug**状态下，计算表达式 **expression**，如果其值为假（即为**0**），那么它先打印一条出错信息，然后通过调用 **abort** 来终止程序运行。通常用来检查捕捉非法的情况。

3.1.2、顺序栈的类模板

压栈

```
template<typename T>
void Stack<T>::Push(const T &data)
{
   assert(!IsFull());
   elements[++top]=data;
}
```

栈满则退出程序

栈顶指针先加1，元素再进栈

出栈

```
template<typename T>
T& Stack<T>::Pop()
{
   assert(!IsEmpty());
  return elements[top--];
}
```

栈已空则退出程序

返回栈顶元素，top减1

3.1.2、顺序栈的类模板

```
template<typename T>
T &Stack<T>::GetElem(int i) const
{
   assert(i<=top&&i>=0);
  return elements[i];
}
```

超出范围则退出程序

返回指定元素，top不变

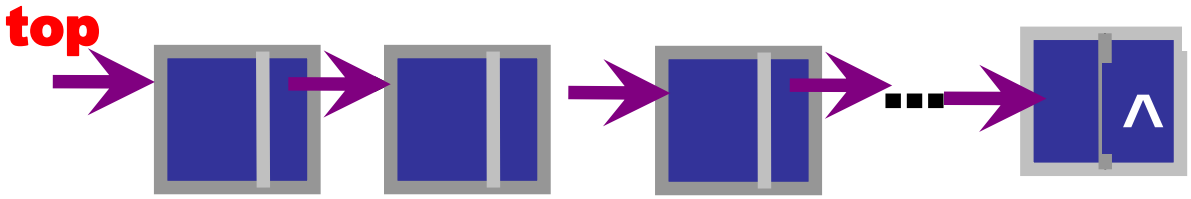
```
template<typename T>
void Stack<T>::PrintStack() const
{
  for(int i=0; i<=top; i++)       cout<<elements[i]<<'\t';
  cout<<endl;
}
```

3.1.2、顺序栈的类模板

```
int main(){
        int i,a[10]={0,1,2,3,4,5,6,7,8,9},b[10];
        Stack<int> istack(10);
        for(i=0;i<10;i++) istack.Push(a[i]);
        if(istack.IsFull()) cout<<"栈满"<<endl;
        istack.PrintStack();
        for(i=0;i<10;i++) b[i]=istack.Pop();
        if(istack.IsEmpty()) cout<<"栈空"<<endl;
        for(i=0;i<10;i++) cout<<b[i]<<'\t';
        cout<<endl;
        istack.Pop();
        return 0;
}
```

代码示例

3.1.3 、链栈的类模板



链栈 （无表头链表）

链栈的结点类模板：

```
template<typename T> class Node
{
        T info;
        Node<T> *link;
public:
        Node(T data=0,Node<T> *next=NULL)
        {       info=data;   link=next;     }
        friend class Stacklink<T>;
};
```

东南大学 信息科学与工程学院
王文博 2026

3.1.3 、链栈的类模板

```
template<typename T>class Stacklink
{
        Node<T> *top;                          //栈顶指针
public:
        Stacklink() {top=NULL;}
        ~Stacklink();                          //析构函数
        void Push(const T &data);              //压栈
        T  Pop();                               //弹出
        T & GetTop() const;                                  //取栈顶元素
        void MakeEmpty();                      //清空栈
        bool IsEmpty() const{  return top==NULL;  }
};
```

```
template<typename T> T & Stacklink<T>::GetTop() const
{
   assert (!IsEmpty());
   return top->info;  }
```

取栈顶

东南大学 信息科学与工程学院
王文博 2026

3.1.3、链栈的类模板

```
template<typename T>
 void  Stacklink<T>::Push(const T &data)
{
        top=new Node<T> (data,top);
}
```

压栈

```
template<typename T> T Stacklink<T>::Pop()
{
        assert(!IsEmpty());
        Node<T> *temp=top;
        T data=temp->info;
        top=top->link;
        delete temp;
        return data;
}
```

弹出

By 王文博

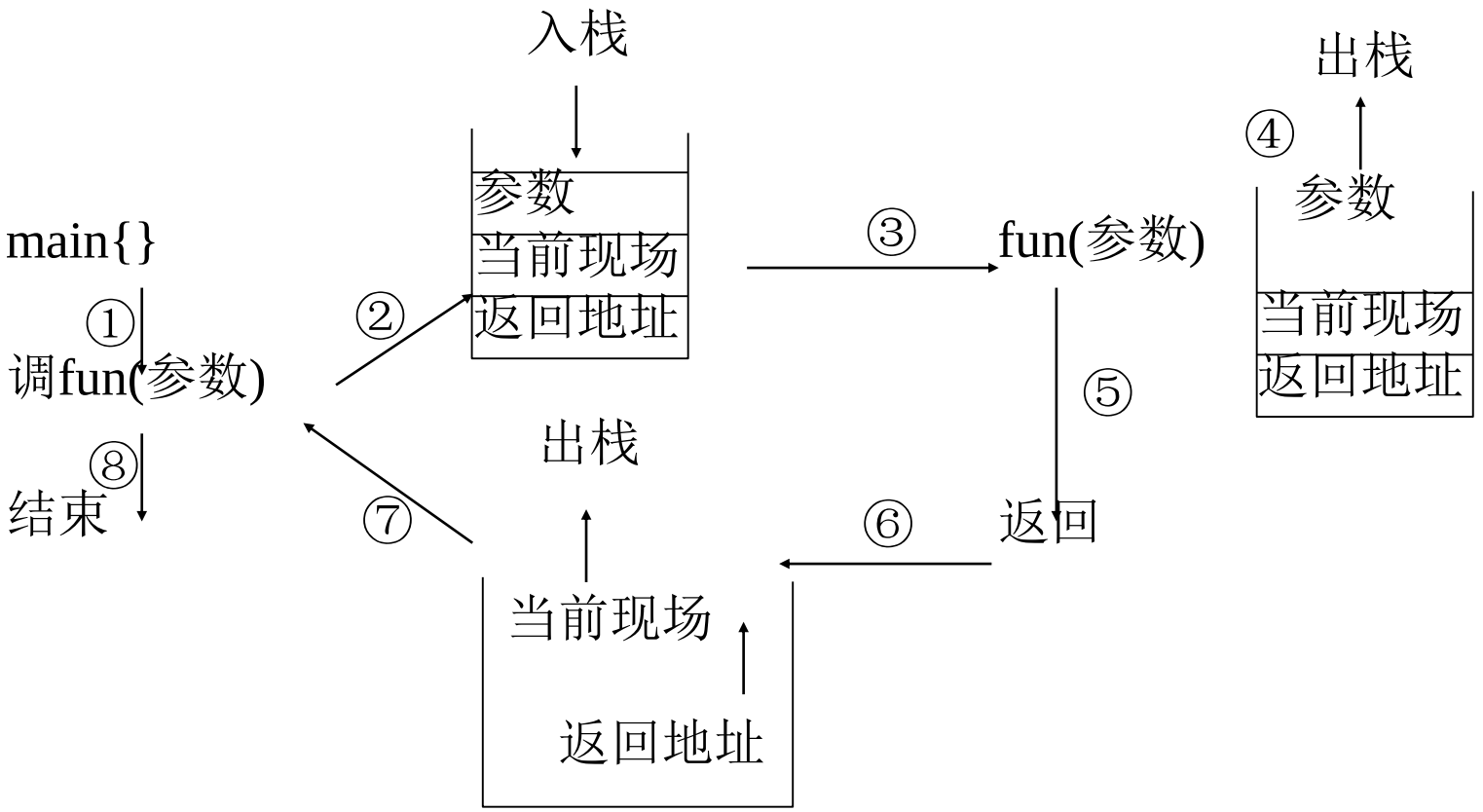
顺序栈和链栈比较

- 栈的逻辑功能一样（逻辑上一样）
- 顺序栈可以随机访问其中的元素，而链栈只能顺序访问（物理结构不同）
- 顺序栈：先开一定大小内存空间，执行起来简单，速度快，可能溢出；链栈：内存空间随用随开，不会溢出，但执行复杂，速度慢。

东南大学 信息科学与工程学院
王文博 2026

3.1.4 、栈的应用-1

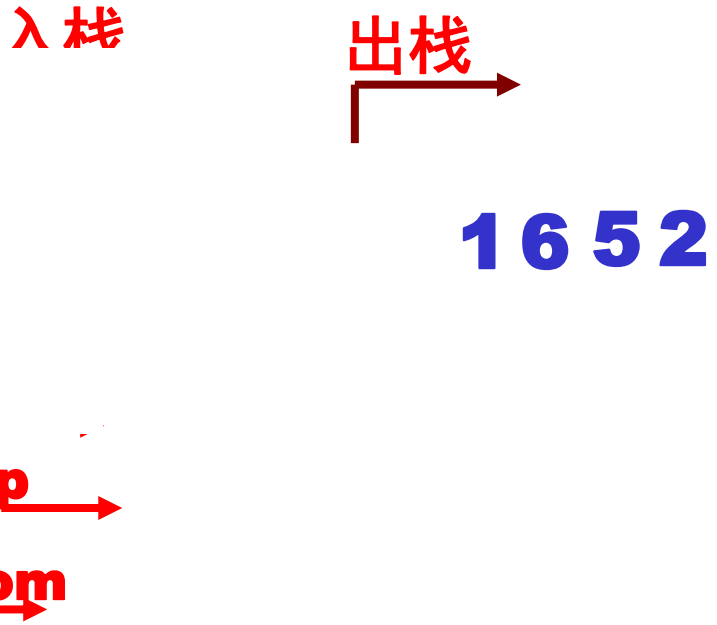
函数的调用



3.1.4 、 栈的应用-2

十进制整数转换为R进制数

$(938)_{10}$ →八进制数



3.1.4 、栈的应用-2

十进制整数转换为R进制数

```
void Decchange(int digital,int R)
{
        Stack<char> istack(10);
        while(digital)
        {
                int res=digital%R;
                char cres=res<9?res+'0':(res-10)+'A';
                istack.Push(cres);
                digital/=R;
        }
        cout<<"转换为"<<R<<"进制为： ";
        while(!istack.IsEmpty()) cout<<istack.Pop();
}
```

3.1.4 、栈的应用-3

字符匹配

在C++语言中有一些符号({ }、 [ ])是成对匹配出现的，利用栈可以实现类似编译器括号是否匹配的能力。

算法思路：

- 从第一个字符开始扫描，当遇见普通字符时忽略，当遇见左符号时压入栈中；
- 当遇见右符号时从栈中弹出栈顶符号　进行匹配
- 如匹配成功：继续读入下一个字符

  若匹配失败：立即停止，并报错

若所有字符扫描完毕，且栈为空，表明所有符号匹配

匹配失败或所有字符扫描完毕但栈非空，则存在不匹配

3.1.4 、栈的应用-4

表达式的计算

两个栈：数栈和运算符栈

从左向右扫描表达式，遇到操作数压数栈，遇到运算符与运算符栈的栈顶运算符比较优先级，若新的运算符优先级高或者运算符栈为空，压栈，否则栈顶运算符出栈，与数栈出栈的两个数据运算，结果压数栈，继续扫描，直到遇到 “=” 。

3.1.4 、栈的应用-4

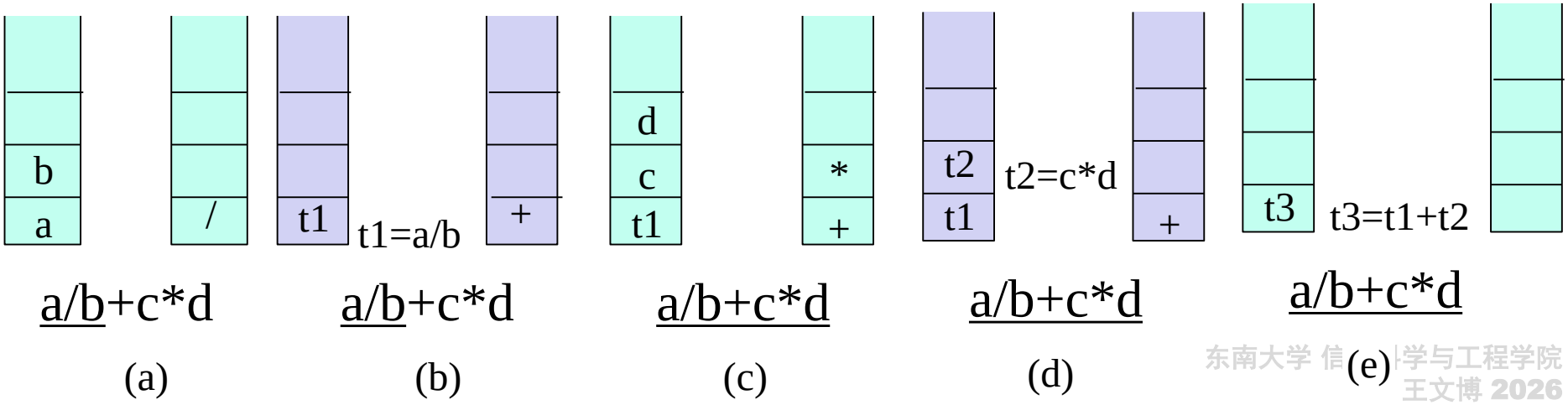
表达式的计算

两个栈：数栈和运算符栈

代码示例

```
class Calculator{
        Stacklink<int> Nstack;
        Stacklink<char> Ostack;
public:
        Calculator(){};
        void Cal();
        void GetTwoNum(int &Num1,int &Num2);
        void Compute(char Opr);
        void Clear();
};
```

a/b+c*d=



东南大学 信息科学与工程学院
王文博 2026

3.1.4 、 栈的应用-5

中缀表达式变后缀表达式

算法思路：

- 从左往右遍历中缀表达式中的数字和符号
- 如果是数字，则直接输出
- 对于符号：

  * 左括号：直接进栈

  * 运算符号：与栈顶元素的进行优先级比较

    若栈顶符号优先级低，运算符号进栈（默认栈顶元素是左括号，左括号优先级最低）

    若栈顶符号优先级高： 将栈顶符号弹出并输出，之后进栈

  * 右括号：将栈顶元素弹出并输出，直到匹配左括号（左括号也要弹出不输出）

- 遍历结束。
- 将栈中所有符号弹出并输出

3.1.4 、 栈的应用-5

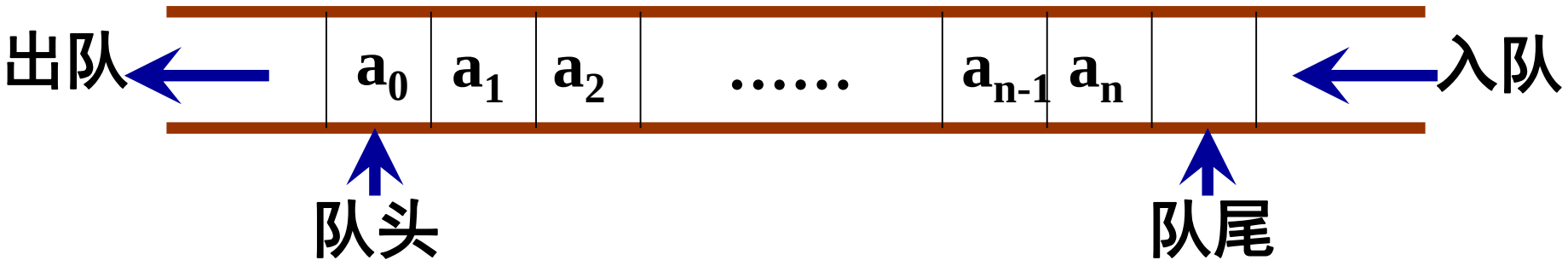
后缀表达式计算思路

算法思路：

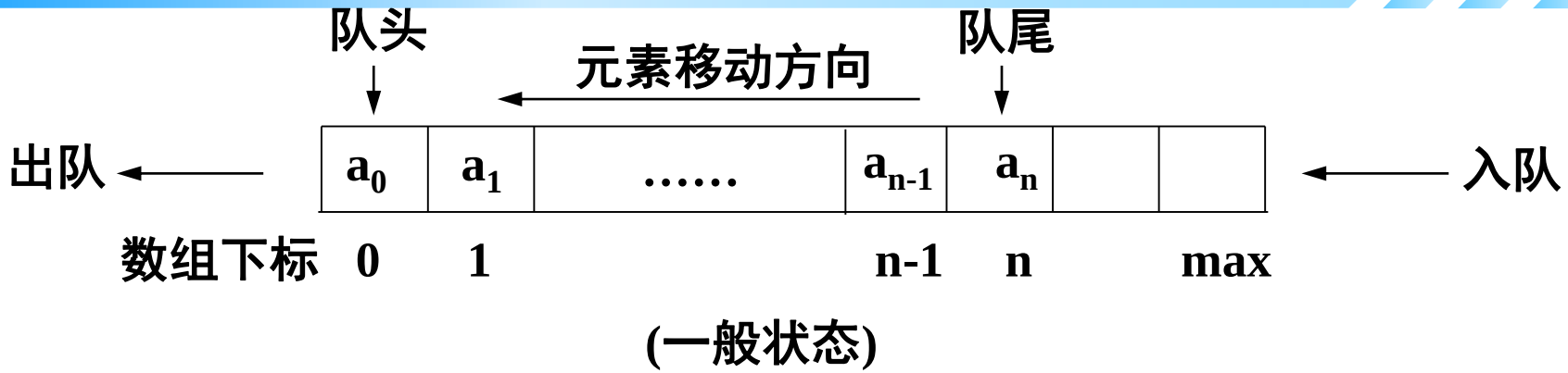
- 遍历后缀表达式中的数字和符号
- 对于数字，直接进栈
- 对于符号：
  - * 从栈中弹出右操作数
  - * 从栈中弹出左操作数
  - * 根据符号进行计算
  - * 将运算结果压入栈中
- 遍历结束。
- 成功：栈中的唯一数字为计算结果

3.2 、 队列

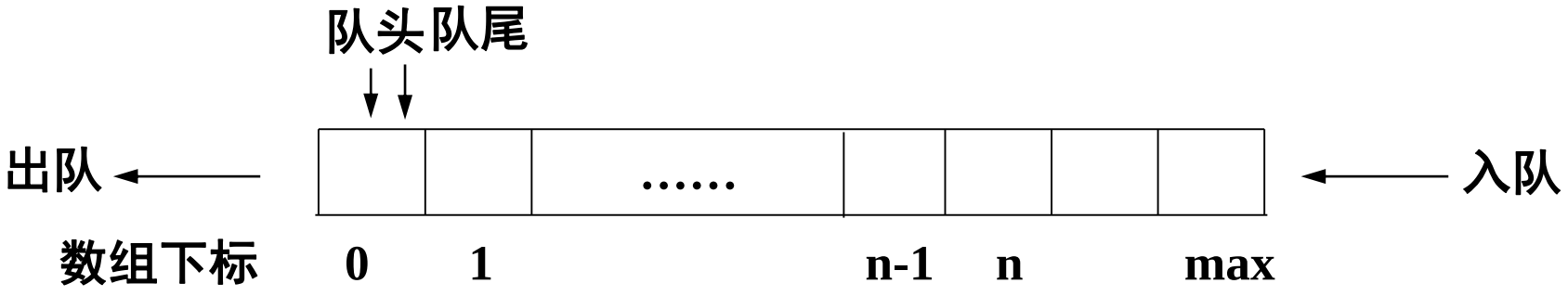
- 只允许在表的一端插入，而在另一端删除。
- 先进先出FIFO(First In First Out)
- 允许插入的一端称为队尾(rear)，允许删除的一端叫做队头(front)。
- 每次在队尾加入新元素，加入称为进队，删除称为出队。



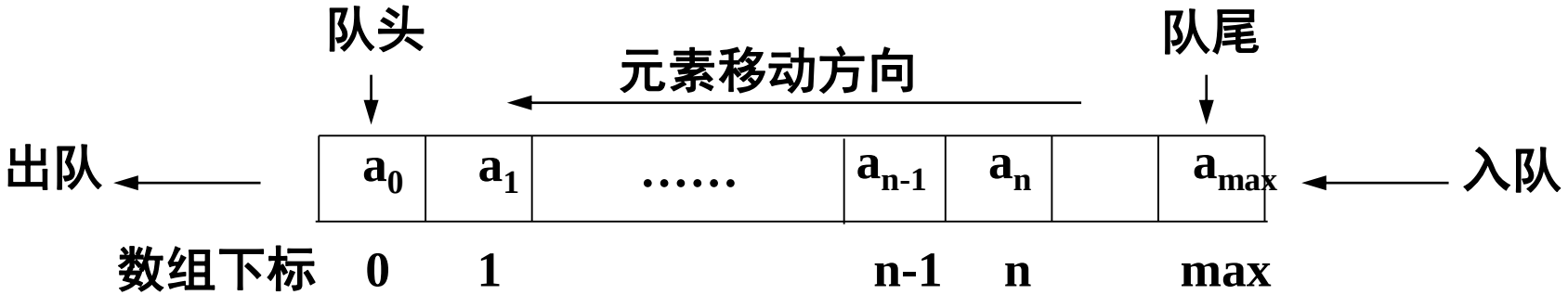
3.2.1、队列-顺序表



(一般状态)



(队空状态)

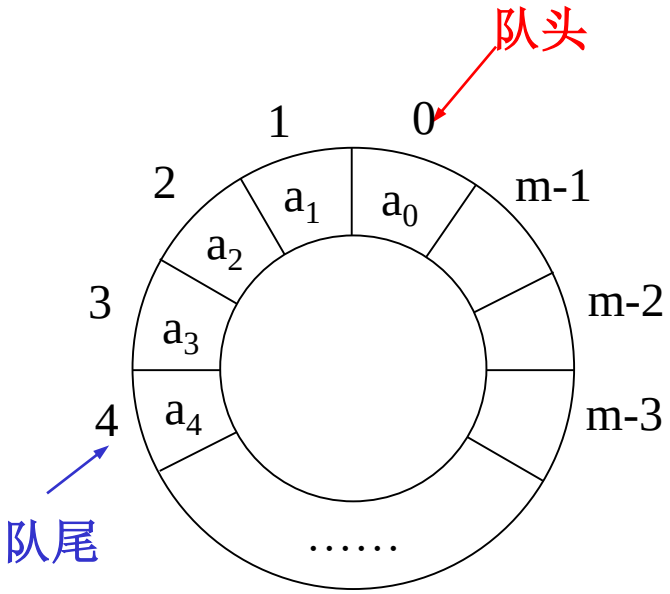


(队满状态)

东南大学 信息科学与工程学院
王文博 2026

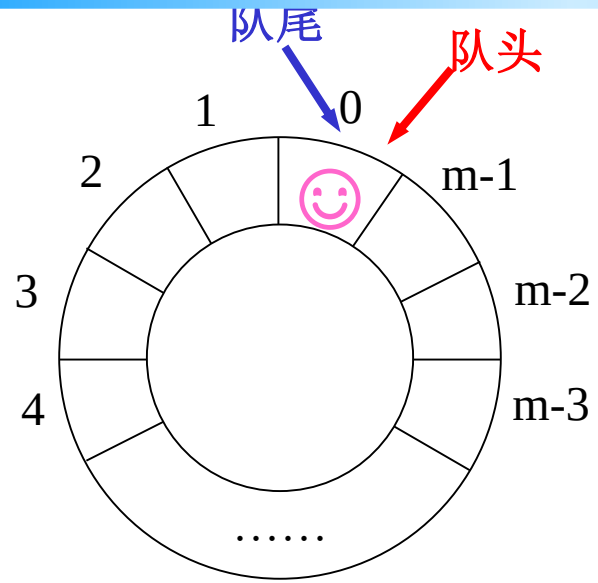
3.2.1 、队列-顺序表 循环队列

在想象中将数组弯曲成环形，元素出队时，后继元素不移动，每当队尾达到数组最后一个元素时，便再回到数组开头。



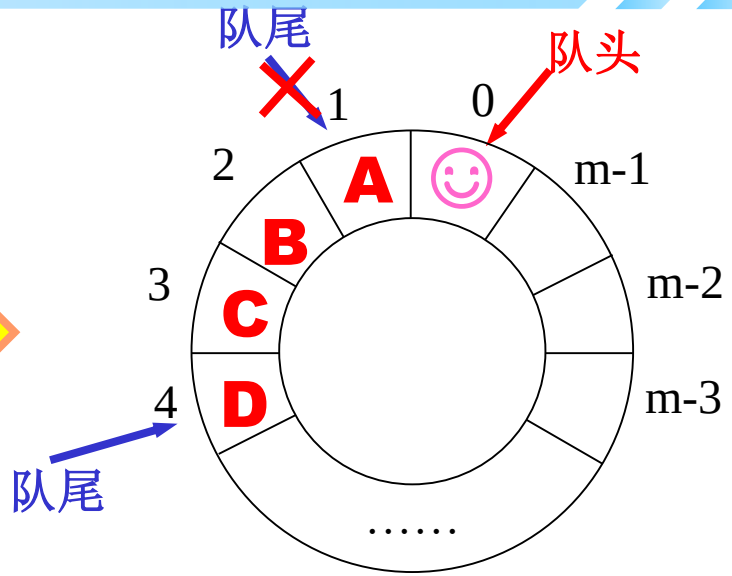
顺序循环队列

3.2.1、队列-顺序表 循环队列

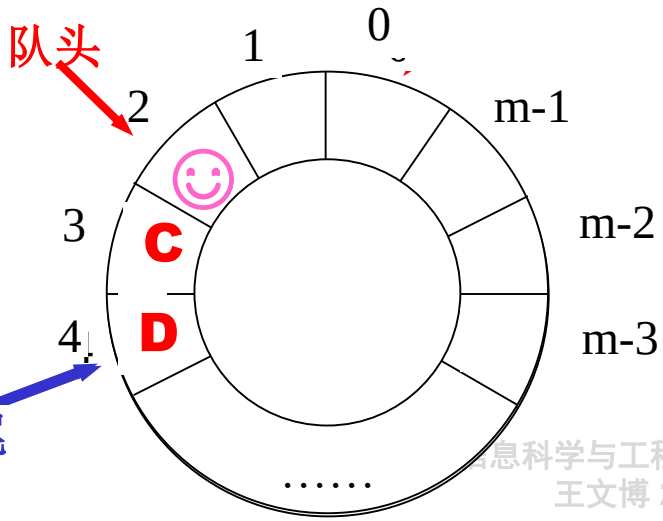


队空状态 元素个数=0

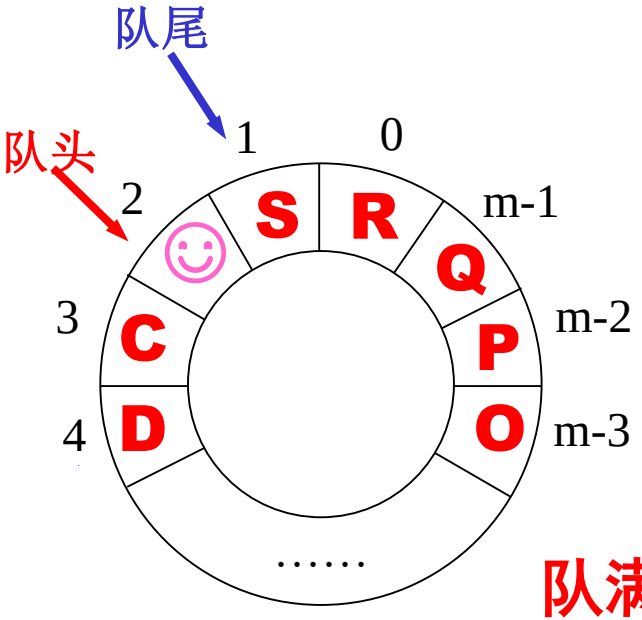
进队



出队



进队



队满

3.2.1、队列-顺序表 循环队列类模板

```
template <typename T>class Queue
{
        int rear,front;     //队尾与队头
        T *elements;        //存放队列元素的容器
        int maxSize;        //队列最多可容纳元素个数+1
public:
        Queue(int ms=18);
         ~Queue(){delete[] elements;}
         bool IsEmpty() const {return front==rear; }
        bool IsFull() const
        {  return (rear+1)%maxSize==front;}
        int Length() const
        {    return  (rear-front+maxSize)%maxSize;}
        void EnQue(const T &data);
        T DeQue();
        T GetFront();
        void MakeEmpty(){front=rear=0;}
};
```

判队空

判队满

求队中元素数，注意求余算法

进队

出队

取队头数据

队置空（初始态）

与工程学院 王文博 2026

3.2.1、队列-顺序表 循环队列类模板

构造函数

```
template<typename T>
Queue<T>::Queue(int ms)
{
        maxSize=ms;
        elements=new T[maxSize];
        rear=front=0;
        assert(elements!=NULL); //分配成功
}
```



3.2.1、队列-顺序表 循环队列类模板

进队

```
template<typename T>
void Queue<T>::EnQue(const T &data)
{
    assert(!IsFull());
    rear= (rear+1)%maxSize
    elements[rear]=data;
}
```

队列不满才能进队

队尾指针加1



3.2.1、队列-顺序表 循环队列类模板

出队

```
template<typename T>T Queue<T>::DeQue()
{
        assert(!IsEmpty());
        front= (front+1)%maxSize
      return elements[front];
}
```

队头指针加1



3.2.1 、队列-顺序表 循环队列类模板

取队头元素

```
template<typename T>T Queue<T>::GetFront()
{
        assert(!IsEmpty());
        return elements[(front+1)%maxSize];
}
```

注意加1才能返回队头数据

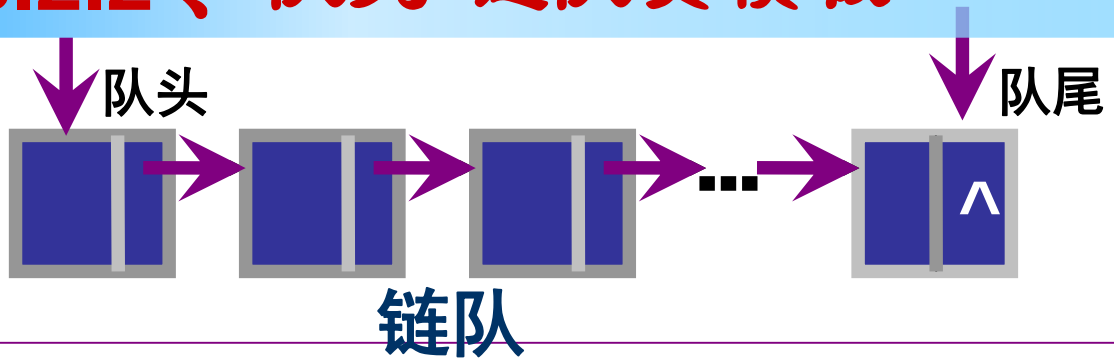


3.2.1、队列-顺序表 循环队列类模板

代码示例

```
int main()
{
    int i;
    Queue<char> que;     //缺省为18元素队列，可用17
     char str1[]="abcdefghijklmnop";  //17个元素(含\0)
     que.MakeEmpty();
    for(i=0;i<17;i++) que.EnQue(str1[i]);
    if(que.IsFull()) cout<<"队满";
    cout<<"共有元素： "<<que.Length()<<endl;
    for(i=0;i<17;i++) cout<<que.DeQue();  //先进先出
     cout<<endl;
    if(que.IsEmpty()) cout<<"队空";
    cout<<"共有元素： "<<que.Length()<<endl;
    return 0;
}
```

3.2.2 、队列-链队类模板



链队

```
template<typename T>class Queuelink;
template<typename T>class Node
{
        T info;
        Node* link;
public:
        Node(T data = 0, Node* next = NULL);
        friend class Queuelink<T>;
};
```

结点模板

3.2.2 、队列-链队类模板

链队类模板

```
template<typename T>class Queuelink
{
        Node<T> * front, * rear;
public:
        Queuelink() { rear = front = NULL; }
        ~Queuelink();
        bool IsEmpty() { return front == NULL; }
        void EnQuelink(const T& data);    //入队
        T DeQuelink();                    //出队
        T GetFront();
        int Length();
        void Empty();
};
```

3.2.2 、队列-链队类模板

```
template<typename  T>Queuelink<T>::~Queuelink()
        {    Empty(); }
```

```
template<typename T>
void Queuelink<T>::Empty()
 {
        Node<T>* temp;
        while (front != NULL)
         {
                temp = front;
                front = front->link;
                delete temp;
        }
}
```

3.2.2 、队列-链队类模板

```
template<typename T>
void Queuelink<T>::EnQuelink(const T& data)
 {
        if (front == NULL)
          front = rear = new Node<T>(data, NULL);
        else
         rear = rear->link = new Node<T>(data, NULL);
}
```

```
template<typename T>T Queuelink<T>::DeQuelink()
 {
        assert(!IsEmpty());
        Node<T>* temp = front;
        T data = temp->info;
        front = front->link;
        delete temp;
        return data;
}
```

3.2.2 、队列-链队类模板

```
template<typename T>T Queuelink<T>::GetFront() {
        assert(!IsEmpty());
        return front->info;
}
```

```
template<typename T>int Queuelink<T>::Length() {
        int count = 0;
        Node<T>* temp=front;
        while (temp!= NULL)
        {
                temp = temp->link;
                count++;
        }
        return count;
}
```